UADE

SISTEMA DE GESTIÓN DE **CLÍNICA ODONTOLÓGICA**

Paradigma Orientado a Objetos — **Entrega 2**

Presentado por:

Aranda, Fabian

Zorro, Brayan

Soto, Christian

Pabón, Gabriel

ÍNDICE

CONTENIDO DEL INFORME

Introducción 1

Arquitectura en Capas 2

a. Capa de Dominio *2.1*

b. Capa de Repositorios *2.2*

c. Capa de Servicios *2.3*

d. Capa de Controladores *2.4*

e. Capa de Presentación *2.5*

Patrones GRASP Aplicados 3

Principios SOLID Demostrados 4

Conclusión de la Entrega 5

INTRODUCCIÓN

El presente documento detalla las decisiones de diseño y arquitectura correspondientes a la **Entrega 2** del sistema "Sonrisa Feliz". Sobre la base del modelo de dominio construido en la entrega anterior, esta iteración incorpora una **arquitectura en capas** completa: repositorios en memoria, servicios con lógica de negocio, controladores que orquestan el flujo y una interfaz interactiva por consola.

Cada decisión de diseño fue guiada por los patrones **GRASP** (Controller, Information Expert, Creator, Low Coupling y High Cohesion) y los principios **SOLID**, en particular SRP y OCP, buscando que cada clase tenga una única razón de cambio y que el sistema sea extensible sin necesidad de modificar código existente.

El resultado es un sistema funcional que acepta operaciones CRUD completas sobre pacientes, odontólogos y turnos, con validaciones de negocio robustas y manejo explícito de errores mediante excepciones personalizadas del dominio.

ARQUITECTURA EN CAPAS

El sistema se organiza en cinco capas con dependencia unidireccional de arriba hacia abajo, sin que ninguna capa inferior conozca a las superiores, garantizando bajo acoplamiento:

Capa	Paquete	Responsabilidad
Presentación	`presentacion.consola`	I/O con el usuario (Scanner, menús)
Controladores	`controlador`	Recibe pedidos de la UI, delega a servicios
Servicios	`servicio`	Lógica de negocio y validaciones
Repositorios	`repositorio`	Persistencia en memoria (HashMap / ArrayList)
Dominio	`modelo`	Entidades puras sin dependencias externas

2.1 CAPA DE DOMINIO

Sin cambios estructurales respecto a la Entrega 1. Se corrigió la asignación de IDs en **Persona** y **Turno** mediante contadores estáticos auto-incrementales (mismo patrón ya presente en *Domicilio*), garantizando que todo objeto tenga un identificador único desde su construcción.

2.2 CAPA DE REPOSITORIOS

Se definió la interfaz genérica **IRepositorio<T>** con los cinco métodos CRUD: `guardar`, `buscarPorId`, `buscarTodos`, `actualizar` y `eliminar`. Cada entidad tiene su implementación concreta:

- **RepositorioPaciente** — `HashMap<Long, Paciente>` con búsqueda adicional por DNI.
- **RepositorioOdontologo** — `HashMap<Long, Odontologo>` con búsqueda por matrícula.
- **RepositorioTurno** — `ArrayList<Turno>` con filtros por paciente, odontólogo, fecha y estado.

2.3 CAPA DE SERVICIOS

Los servicios encapsulan toda la lógica de negocio y son el único punto donde se lanzan excepciones del dominio. Cada servicio recibe sus repositorios por constructor (*inyección de dependencias manual*), lo que facilita el reemplazo futuro de la estrategia de persistencia:

- **ServicioPaciente** — valida DNI no duplicado al registrar; impide eliminar pacientes con turnos futuros; lista en orden alfabético.
- **ServicioOdontologo** — valida matrícula no duplicada; impide eliminar odontólogos con turnos futuros.
- **ServicioTurno** — verifica disponibilidad del odontólogo antes de reservar; coordina la actualización de la *Agenda* y la lista de turnos del *Paciente*; ofrece múltiples vistas filtradas.

2.4 CAPA DE CONTROLADORES

Inspirada en el **patrón GRASP Controller**, esta capa actúa como fachada entre la presentación y los servicios. **ControladorPaciente**, **ControladorOdontologo** y **ControladorTurno** reciben llamadas ya procesadas de los menús y las redirigen al servicio correspondiente, sin contener lógica de negocio propia.

> **Decisión de diseño:** separar controladores de servicios permite que en la Entrega 4 (GUI Swing) los paneles gráficos reutilicen exactamente los mismos controladores sin modificar la capa de servicios, cumpliendo el principio OCP.

2.5 CAPA DE PRESENTACIÓN

Se dividió en cuatro clases para respetar SRP:

- **ConsolaUtils** — helpers de entrada (`leerTexto`, `leerFecha`, `leerHora`, etc.) compartidos por todos los menús.
- **MenuPacientes / MenuOdontologos / MenuTurnos** — cada uno gestiona exclusivamente su entidad.
- **MenuPrincipal** — solo contiene la navegación raíz y delega a los submenús.

PATRONES GRASP APLICADOS

CONTROLLER

ControladorPaciente ControladorOdontologo ControladorTurno

Los tres controladores son el punto de entrada del sistema para cada entidad. Reciben los eventos de la interfaz (acciones del usuario), coordinan la respuesta y delegan el trabajo al servicio correspondiente. No toman decisiones de negocio: esa responsabilidad recae exclusivamente en la capa de servicios.

INFORMATION EXPERT

Agenda Paciente

El método `estaDisponible(fecha, hora)` vive en **Agenda** porque es quien posee la colección de turnos del odontólogo — el experto en esa información. De igual forma, `esFuturo()` vive en **Turno** porque solo él conoce su fecha y hora. Ningún servicio ni controlador duplica esa lógica.

CREATOR

ServicioPaciente ServicioTurno

ServicioPaciente crea instancias de *Paciente* porque es quien valida los datos previos a la creación (DNI único). **ServicioTurno** crea instancias de *Turno* porque es quien verifica la disponibilidad y coordina la actualización de *Agenda* y *Paciente*. Los constructores de entidades no contienen lógica de validación; esa responsabilidad se delega al creador correcto.

LOW COUPLING / HIGH COHESION

Cada capa solo conoce a la inmediatamente inferior. *MenuPacientes* solo conoce a *ControladorPaciente*; el controlador solo conoce a *ServicioPaciente*; el servicio solo conoce a los repositorios. Las entidades del dominio (`modelo/`) no tienen ninguna dependencia externa, maximizando su reusabilidad y testeabilidad.

PRINCIPIOS SOLID DEMOSTRADOS

SRP — SINGLE RESPONSIBILITY PRINCIPLE

Cada clase tiene una única razón de cambio. Ejemplos concretos:

- **ConsolaUtils** solo maneja lectura de entrada por teclado; si cambia el formato de fecha, solo se modifica esta clase.
- **RepositorioPaciente** solo sabe guardar y buscar pacientes; no conoce reglas de negocio.
- **ServicioPaciente** solo aplica reglas de negocio sobre pacientes; no sabe cómo se almacenan ni cómo se muestran.

OCP — OPEN/CLOSED PRINCIPLE

La interfaz **IRepositorio<T>** permite extender la estrategia de persistencia sin modificar los servicios. En la Entrega 3, cuando se implemente persistencia en archivos CSV, bastará con crear *RepositorioPacienteCSV* que implemente la misma interfaz; los servicios no requerirán ningún cambio.

CONCLUSIÓN DE LA ENTREGA

La Entrega 2 transforma el modelo de dominio de la Entrega 1 en un sistema funcional de cinco capas. La introducción de la interfaz *IRepositorio<T>* sienta las bases para la persistencia en archivos de la Entrega 3, mientras que la separación en controladores independientes facilita la migración a la interfaz gráfica Swing de la Entrega 4 sin reescribir lógica de negocio.

Los patrones GRASP y los principios SOLID no fueron aplicados como requisito formal, sino como guía real de cada decisión: desde la elección de dónde crear un objeto hasta cómo dividir una clase que concentraba demasiadas responsabilidades.